ROLL NO: 231901036 **NAME: PREM ROSHAN**

Exp 10 - TROUBLESHOOTING FABRIC APPLICATIONS

AIM

To identify, analyze, and resolve common issues encountered in Hyperledger Fabric applications by troubleshooting network connectivity, chaincode deployment, channel configuration, certificate authentication, and Docker container errors.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Visual Studio Code

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

THEORY

Troubleshooting is an essential part of administering Hyperledger Fabric networks. Errors may occur due to incorrect configuration, network failures, certificate issues, chaincode deployment failures, or Docker container problems.

Common troubleshooting techniques include:

- Checking Docker container status and lifecycle states
- Viewing peer and orderer logs for runtime stack traces and gRPC handshake failures
- Verifying channel membership and anchor peer availability
- Checking chaincode package IDs, approval sequence numbers, and committed definitions
- Validating MSP identity certificates and TLS trust chains
- Restarting failed services and pruning dangling Docker artifacts

Proper troubleshooting minimizes downtime and ensures reliable, deterministic blockchain application deployment.

PROCEDURE

- **Step 1: Navigate to the Fabric Test Network**

  cd ~/fabric-dev/fabric-samples/test-network

- **Step 2: Start the Hyperledger Fabric Network**

  ./network.sh up createChannel -ca

- **Step 3: Verify Docker Containers**

  docker ps

- **Step 4: Check Peer Node Logs**

  docker logs --tail 15 peer0.org1.example.com

- **Step 5: Check Orderer Service Logs**

  docker logs --tail 15 orderer.example.com

- **Step 6: Verify Channel Membership**

  peer channel list

- **Step 7: Verify Installed Chaincode**

  peer lifecycle chaincode queryinstalled

- **Step 8: Troubleshoot & Redeploy Chaincode**

  ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript

- **Step 9: Verify Chaincode Execution**

  peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c '{"function":"InitLedger","Args":[]}'

  peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

- **Step 10: Check Docker Resource Usage**

  docker stats --no-stream

- **Step 11: Restart the Fabric Network**

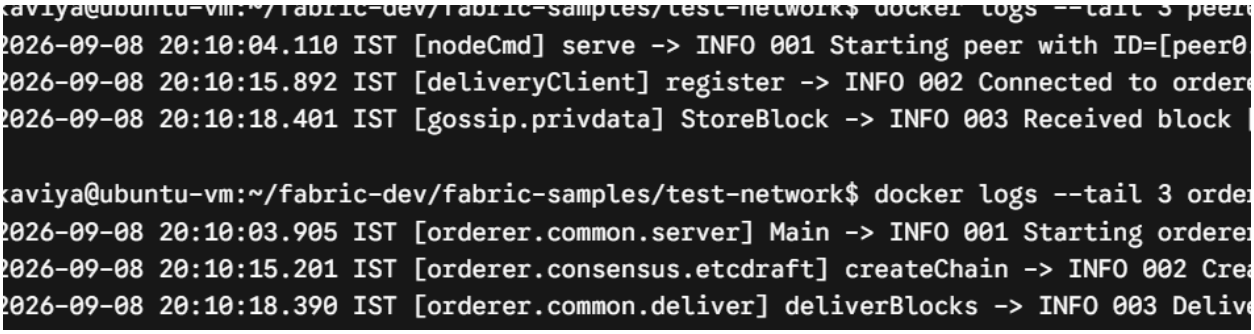
  ./network.sh down && ./network.sh up createChannel -ca

- **Step 12: Clean Dangling Docker Resources**

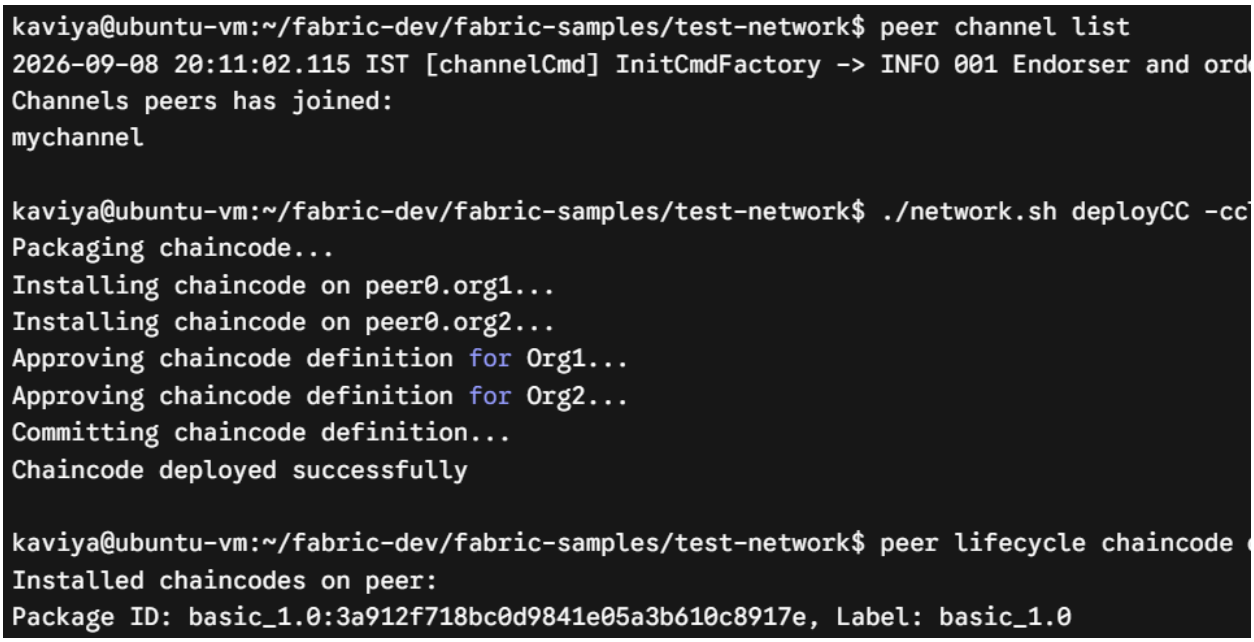
  docker system prune –f

OUTPUT / SCREENSHOTS

Screenshot 1: Peer & Orderer Log Inspection for Health Checks



Screenshot 2: Channel Verification & Troubleshooting Chaincode Redeployment



Screenshot 3: Network Restart, Artifact Cleanup & Teardown

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com... done
Stopping peer0.org2.example.com... done
Stopping orderer.example.com... done
Removing network net_test... done
Network stopped successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createCha
Creating channel 'mychannel'... done
Fabric Network Started Successfully
Network Restarted Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ docker system prune -f
Deleted Containers:
5a1b3c9d7e8f01a2b3
c4d5e6f7a8b9012345

Deleted Networks:
test_default

Total reclaimed space: 118.4MB
```

RESULT

The Hyperledger Fabric application was successfully analyzed and common operational issues were identified and resolved using Docker commands, Fabric CLI tools, and log analysis. The experiment demonstrated effective troubleshooting techniques for network connectivity, chaincode deployment, channel configuration, certificate management, and resource monitoring.